Lenses & Optics

Lenses · Update nested city in immutable Person/Address · Build with Waqar

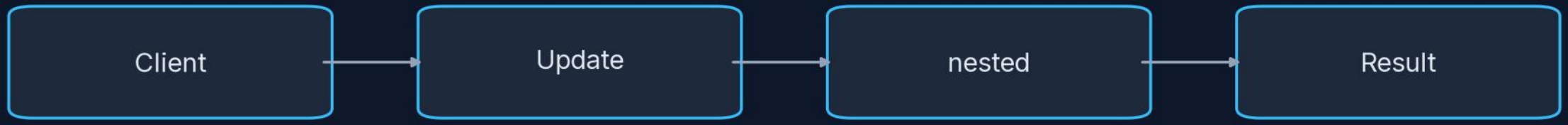
Today's Hook

- Problem: Update nested city in immutable Person/Address
- Pattern: Lenses — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Update nested city in immutable Person/Address
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Lenses: Update nested city in immutable Person/Address

Real-World Scenario

- Running example: Update nested city in immutable Person/Address
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```
package com.buildwithwaqar.patterns.functional.lens.problem;

import java.util.*;

public final class NaivePerson {
  public static String move(String personJson, String city){ return personJson.replace("Old", city); }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.functional.lens.problem;

import java.util.*;
import java.util.function.*;

record Address(String city, String zip){}
record Person(String name, Address address){}
record Lens<S,A>(Function<S,A> get, BiFunction<S,A,S> set){
  A getA(S s){ return get.apply(s); }
  S setA(S s, A a){ return set.apply(s,a); }
  <B> Lens<S,B> andThen(Lens<A,B> other){
    return new Lens<>(s -> other.get.apply(get.apply(s)), (s,b) -> set.apply(s, other.set.apply(get.apply(s), b)));
  }
}
final class PersonLenses {
  static final Lens<Person,Address> address = new Lens<>(Person::address, (p,a)-> new Person(p.name(), a));
  static final Lens<Address,String> city = new Lens<>(Address::city, (a,c)-> new Address(c, a.zip()));
  static final Lens<Person,String> personCity = address.andThen(city);
}
```

Runnable main

```
package com.buildwithwaqar.patterns.functional.lens.problem;
import java.util.*; import java.util.function.*;
public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== Lenses: nested city update ===");
    System.out.println("--- naive ---"); System.out.println(NaivePerson.move("{city:Old}", "NYC"));
System.out.println("--- lenses ---");
var p=new Person("Ada", new Address("Old", "10001"));
var moved=PersonLenses.personCity.setA(p, "NYC");
System.out.println(p); System.out.println(moved);
System.out.println("city="+PersonLenses.personCity.getA(moved));
  }
}
```

Walkthrough Output

```
=== Lenses: nested city update ===
--- naive ---
{city:NYC}
--- lenses ---
Person[name=Ada, address=Address[city=Old, zip=10001]]
Person[name=Ada, address=Address[city=NYC, zip=10001]]
city=NYC
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Lenses.

Related Patterns

- Contrast Lenses with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Lenses → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.